

Code Explanation

Applying Address Change Transformations for YUYU Process

This is a PySpark function that applies a series of transformations to input DataFrames to generate output DataFrames for the YUYU address change process. The function takes four input DataFrames and a process user ID, and returns two output DataFrames.

Overview of the Code

The code is a PySpark function named ``apply_address_change_transformations`` that performs a series of data transformations on input DataFrames. It joins the input DataFrames with lookup tables, applies various business rules to transform the data, generates sequence numbers, and creates two output DataFrames.

Step 1: Initial Pass-Through and Preparation

This step simply assigns the input ``source_df`` to a new DataFrame ``exp_add_chng_df`` without any transformations.

```
exp_add_chng_df = source_df
```

Step 2: Join with Lookup Tables for Owner Name Enrichment

This step joins the ``exp_add_chng_df`` with two lookup tables, ``yuyu_clnt_df`` and ``yuyuk_cln_df``, on the ``T_TX_BASIC_POLICY_ID`` column. The joins are performed using a left join.

```
enriched_df = exp_add_chng_df.join(
    yuyu_clnt_df,
    exp_add_chng_df["T_TX_BASIC_POLICY_ID"] == yuyu_clnt_df["POL_NO"],
    "left"
)
enriched_df = enriched_df.join(
    yuyuk_cln_df,
    enriched_df["T_TX_BASIC_POLICY_ID"] == yuyuk_cln_df["POL_NO"],
    "left"
)
```

Step 3: Apply Business Transformations

This step applies various business rules to the ``enriched_df`` DataFrame. The transformations include formatting and converting date columns, handling null or empty values, and deriving new columns based on existing ones.

```
transformed_df = enriched_df.withColumn(
    "O_REQUEST_ACCEPT_DATETIME",
    to_timestamp(
        date_format(col("T_TX_REQUEST_REQ_ACC_DATETIME"), "yyyy/MM/dd HH:mm"),
        "yyyy-MM-dd HH:mm"
    )
).withColumn(
    "V T TX DTL ADD CH N TRANS ZIP",
```

```

    when(
      (col("T_TX_DTL_ADD_CH_N_TRANS_ZIP").isNull()) |
      (trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP")) == "") |
      (length(trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"))) == 0),
      col("PPAY_ZIP")
    ).otherwise(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"))
  )
# ... other transformations ...

```

Step 4: Generate Sequence Numbers

This step generates a sequence number for each row in the `transformed\_df` DataFrame based on the `T\_TX\_REQUEST\_REQUEST\_ID` column.

```

window_spec = Window.orderBy("T_TX_REQUEST_REQUEST_ID")
sequence_df = transformed_df.withColumn("SEQUENCE_NUMBER", F.row_number().over(window_spec) - 1)

```

Step 5: Create Output DataFrame for DPAddressChangeYUYU

This step creates a new DataFrame `yuyu\_output\_df` by selecting specific columns from the `sequence\_df` DataFrame and aliasing them to match the required output schema.

```

yuyu_output_df = sequence_df.select(
  col("SEQUENCE_NUMBER"),
  col("T_TX_REQUEST_ORIGIN_REQUEST_ID").alias("ORIGIN_REQUEST_ID"),
  '
  col("T_TX_BASIC_POLICY_ID").alias("POLICY_ID"),
  # ... other columns ...
)

```

Step 6: Create Update DataFrame for STG_E2E_AC_TXDBH_DATA_TGT

This step creates a new DataFrame `staging\_update\_df` by selecting specific columns from the `sequence\_df` DataFrame and aliasing them to match the required output schema.

```

staging_update_df = sequence_df.select(
  col("T_TX_REQUEST_REQUEST_ID"),
  col("T_TX_BASIC_TRANS_ID"),
  col("T_TX_RELATION_TRANS_REL_ID"),
  # ... other columns ...
)

```

The function returns the `yuyu\_output\_df` and `staging\_update\_df` DataFrames.

```

return yuyu_output_df, staging_update_df

```